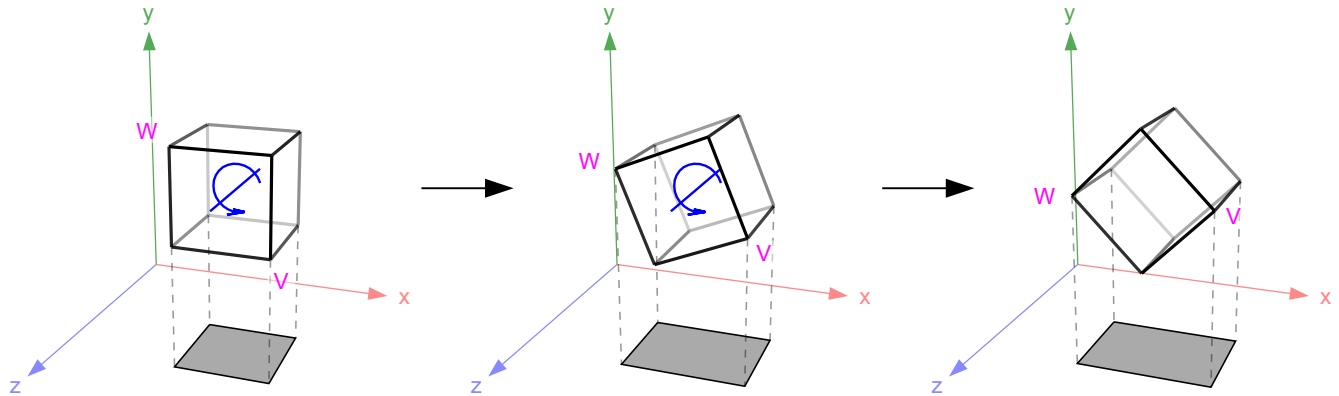


Analysis: Cubic UFO

This is an ad hoc geometry problem with many different solutions.

Test Set 1 (Visible)

Suppose the cube is initially axis-aligned. Let us rotate it about the z-axis by angle t , from $+x$ towards $+y$, and study the shadow:



Key observations:

- The shadow is a rectangle aligned to x- and z-axes, starting out as a square for $t = 0$.
- For $0 \leq t \leq \pi/4$ (45 degrees): z-length = 1 always, and x-length = $V_x - W_x$, where V_x and W_x are x-components of vertices V and W in the figure. Therefore this is really a 2-D problem; we can ignore z!
- The shadow area is $A = 1 \times (V_x - W_x) = 2 V_x$, since $W_x = -V_x$.

For this setup, maximal area is attached when $t = \pi/4$, which corresponds to $V_x = \sqrt{1/2}$, resulting in $A = \sqrt{2} \approx 1.414214$. This exceeds all Test Set 1 inputs, so the setup is sound.

Next, we find the shadow area A as a function of angle t . From basic geometry, $V_x = \sqrt{1/2} \times \cos(t - \pi/4)$. Therefore $A = 2V_x = \sqrt{2} \times \cos(t - \pi/4)$, for $0 \leq t \leq \pi/4$.

Given A , naively we'd invert the formula and get t as sum of $\pi/4$ and $\cos^{-1}(A / \sqrt{2})$. However, to satisfy $0 \leq t \leq \pi/4$, we need the negative branch of $\cos^{-1}$! Therefore the inverse is:

$$t = \pi/4 - |\cos^{-1}(A / \sqrt{2})|.$$

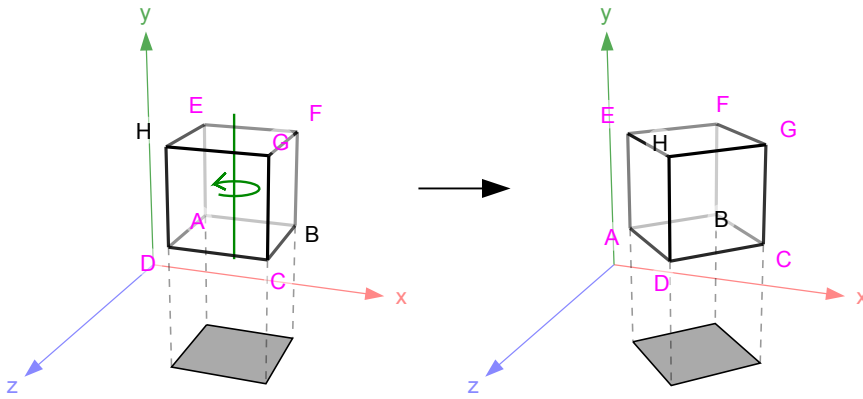
Once t is obtained, the final outputs are the centers of three non-parallel faces. One such face (invariant for all t) is $(0, 0, 1/2)$. The other two can be obtained from rotating $(1/2, 0)$, and $(0, 1/2)$ by angle t , and assigning z to 0. Using the [rotation formula](#) yields $(1/2 \cos(t), 1/2 \sin(t), 0)$ and $(-1/2 \sin(t), 1/2 \cos(t), 0)$.

Test Set 2 (Hidden)

Our solution hinges on the following two crucial observations:

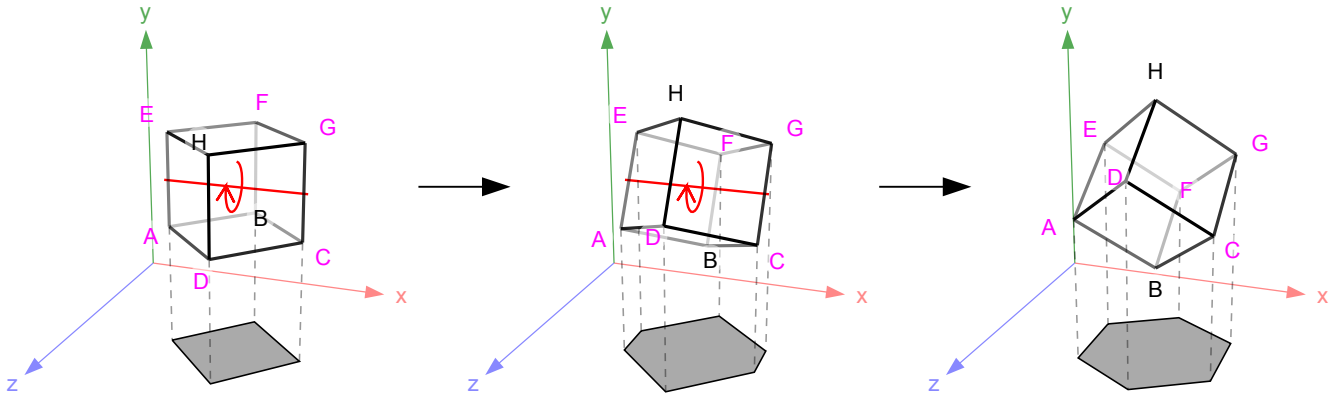
- The cube will cast the smallest possible shadow, which has a square shape, when one of its faces is parallel to the xz-plane.
- The cube will cast the largest possible shadow, which has the shape of a regular hexagon, when one of its vertices is on the y-axis.

To simplify the computations, let's rotate the cube about the y-axis by 45 degrees. (The direction of the rotation does not matter, since the cube would end up in the same orientation either way.) After that, the cube will look like this:



It might not be immediately clear why this simplifies our life, but it will make sense soon!

According to the above observations, the cube currently has the smallest possible shadow. To maximize that shadow, we can rotate the cube about the x-axis from +z towards +y, and bring the vertex H (from our diagram above) onto the y-axis. A useful property of this rotation is that the area of the shadow consistently increases throughout this rotation. Since we start with the smallest possible shadow and continuously rotate until we get the largest possible shadow, we achieve every possible shadow area at some point during this rotation. So, we can use [binary search](#) to figure out the exact angle by which we need to rotate the cube about the x-axis to achieve the desired area.



However, two questions remain:

- If we rotate the cube about the x-axis by a certain angle, what will be the coordinates of the vertices of the cube?
- Given the coordinates of the vertices, how can we calculate the area of the shadow?

Rotating a cube about the x-axis

Notice that, since we are rotating the cube about the x-axis, the x-coordinates of the points will remain the same; only the y- and z- coordinates will change. So, instead of performing rotations in 3-D, we will project the point onto the yz-plane (the $x = 0$ plane) and perform the rotations in 2-D.

For example, suppose that we want to rotate point $P = (P_x, P_y, P_z)$ about the x-axis from +z towards +y by angle t . First, we project the point onto the yz-plane, where it will have coordinates $(0, P_y, P_z)$. We will ignore the x component and treat the point as (P_y, P_z) on a 2-D plane.

Now, rotation about the x-axis by angle t in the indicated direction is equivalent to rotating (P_y, P_z) about $(0, 0)$ by angle t in a clockwise direction. The resulting 2-D point will be

$$(P_y', P_z') = (P_y \times \cos(t) + P_z \times \sin(t), -P_y \times \sin(t) + P_z \times \cos(t)),$$

which in 3-D becomes (P_x, P_y', P_z') . We can get this from the rotation formula, or the complex expression $(P_y + iP_z) \times e^{-it}$.

Shadow area

As H approaches the y-axis, the shadow on the $y = -3$ plane takes the shape of a convex hexagon. More specifically, the vertices of the hexagon are the projection of points D, C, G, F, E, and A onto the $y = -3$ plane. For a point P with coordinates (P_x, P_y, P_z) , the coordinates of its projection onto the $y = -3$ plane are $(P_x, -3, P_z)$.

Now, to find the area of the shadow, we can treat the six projected vertices as if they were on a 2-D plane, with only their x- and z-coordinates. By construction, these already form a [convex hull](#) with vertices properly oriented (otherwise one would need to explicitly compute the convex hull, although shortcuts exist for the special case). This enables us to apply the [standard formula](#) to compute area of a convex polygon. Note that area computation can also be simplified by using symmetry with respect to the z-axis, and apply the trapezoid area formula instead of the general convex polygon.

Once we have a set of coordinates that produces the desired area, we can compute the coordinates of the face-centers of any three non-pairwise-parallel faces, and we have solved the problem. With the setup above, these are simply $\frac{1}{2}(A + C)$, $\frac{1}{2}(C + F)$, and $\frac{1}{2}(F + A)$.

Other approaches

There are many other ways to solve this problem:

- Instead of binary searching, the angle of rotation can be solved directly. In fact, final coordinates can be computed using a closed form without using trig functions!
- We could avoid computing the area of the shadow by using [this amazing cube shadow theorem](#).
- Instead of doing two rotations (one about the y-axis and another about the x-axis), we could rotate the cube about line connecting points (0, 0, 0) and (1, 0, 1), or some other similar axis, to bring a vertex onto the y-axis. The general rotation formula can be found [here](#).

Analysis: Go, Gopher!

Test set 1 (Visible)

For test set 1, we need to prepare all the cells within a grid-aligned rectangle of size at least 20. Before starting to deploy the gopher, let's choose a rectangular target region of size at least 20. We will try to prepare all the cells within this target region. One option is to choose a 4 x 5 target region. We could have chosen 3 x 7, 5 x 5 etc, but it should not be too big. It does not matter where we place this target region in our initial 1000 x 1000 matrix. So let's place it such that one corner of the rectangle is at (1, 1) and the opposite corner is at (4, 5). Here (r, c) refers to the cell in the r-th row and the c-th column of the original matrix. Now the question is: can we come up with a strategy that will prepare all of the cells in this target region, and no other cells?

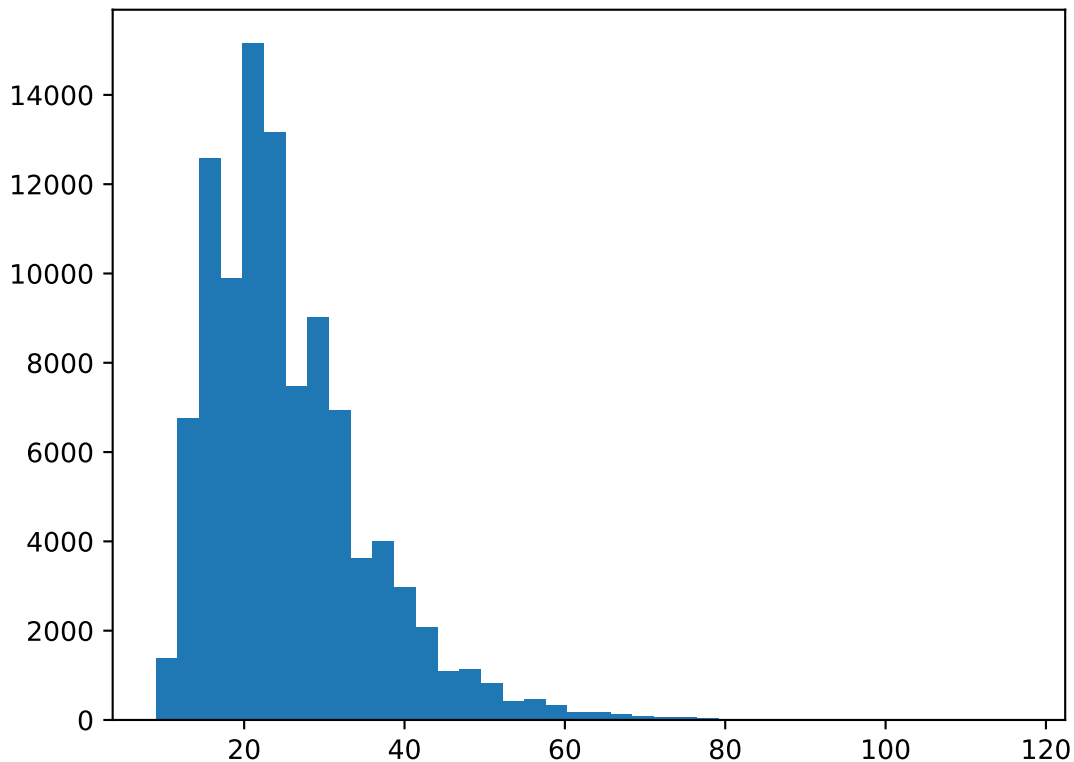
Let's visualize our 4 x 5 target region as follows (row and column numbers are given for convenience):

```
  12345
1  xxxxx
2  x@@@x
3  x@@@x
4  xxxxx
```

We marked the internal cells with @ and the border cells with x. The gopher should not be deployed on the border cells, because that might cause it to prepare a cell outside of our target region. We will only deploy on the internal cells (cells marked with @ in the above picture). We can deploy the gopher 1000 times in total and we have 6 internal cells. Let's deploy the gopher to each of the internal cells $\text{floor}(1000 / 6) = 166$ times. Will this be enough to solve test set 1?

To answer this crucial question, let us compute the probability that after deploying the gopher as described above, the (1, 1) cell is still unprepared. Notice that this cell can only be prepared by deploying the gopher at (2, 2). Every time we do this, it has a 1/9 probability of preparing (1, 1). Looking at it from another angle, every time we do this, it has an 8/9 probability of not preparing (1, 1). So the probability that (1, 1) will not be prepared after 166 deployments to (2, 2) is $(8/9)^{166} = 3.226 \times 10^{-9}$, which is quite small. Realistically, we do not need to worry about this happening to any of the four corners (since the probability of this happening to at least one of the four corners is $1 - (1 - 3.226 \times 10^{-9})^4 = 1.29 \times 10^{-8}$). Other cells are adjacent to more than one internal cell, and thus they are more likely to be prepared than the corners. So this solution is sufficient to pass test set 1.

To convince you further we ran a simulation. We deployed the gopher to the center of a 3 x 3 region until all the nine cells in the region is prepared. We ran this 100000 times and following is the result:



The above figure shows how many of our 100000 simulations (y axis) required each possible number of deployments (x axis). As we can see, out of 100000 simulations the maximum number of times we needed to deploy the gopher was not more than 120. So 166 deployments would be more than enough to prepare an internal cell and all of its surroundings. Moreover, once we have prepared an internal cell and all eight of its neighbors, there is no reason to deploy the gopher there again, so we will almost certainly have more than 166 deployments to use to fill in the last stubborn cell if necessary.

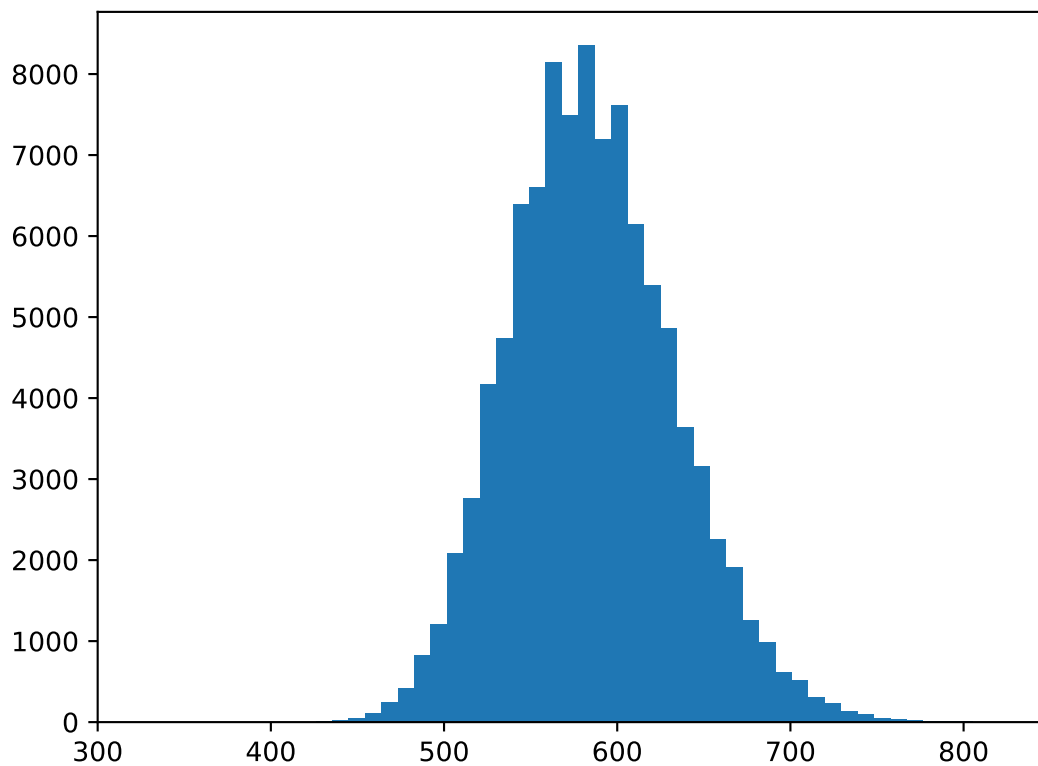
Test set 2 (Hidden)

Now we need to create a rectangular region of area at least 200. If we used the same strategy described above, for, say a 10 x 20 size rectangle, we could make $\text{floor}(1000 / (18 * 8)) = 6$ deployments to each of the internal cells. But then the probability of (1, 1) being not prepared would be: $(8/9)^6 = 0.49327$, which is way too high!

How can we improve this strategy? We can observe that most of the cells can be prepared from a number of locations. For example, the cell (2, 2) can potentially be prepared on a deployment to any of the internal cells around it, or to the cell itself. What if we divide our rectangular region into disjoint 3 x 3 regions and only deploy the gopher to the center cells of those regions? This way each of the cells can be prepared from only one cell. To sum things up, our plan is:

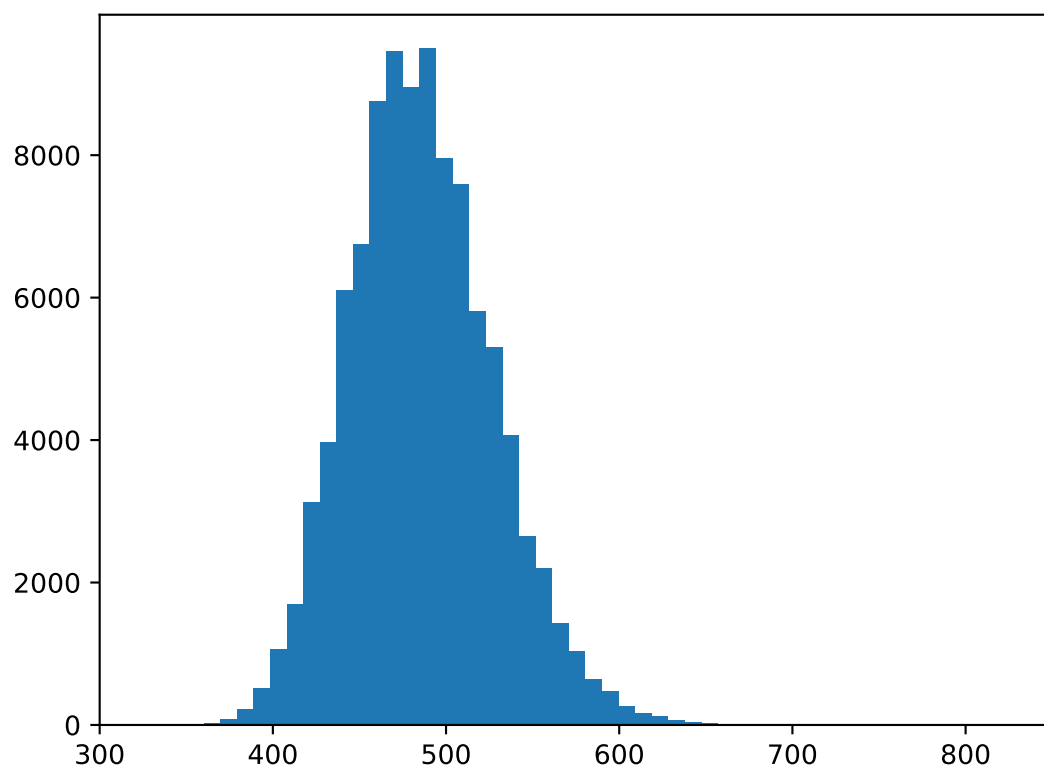
- Select a large enough rectangle, say 3 x 69.
- For convenience, we will place the corner of this 3 x 69 rectangle at (1, 1).
- We will divide our initial 3 x 69 region into $69/3 = 23$ disjoint 3 x 3 regions. That is, we will deploy the gopher only to (2, 2), (2, 5), (2, 8) ... (2, 68): the center cells of those regions.
- We will keep deploying the gopher at (2, 2) until all the cells inside the 3 x 3 grid centered on (2, 2) are prepared.
- Then we will deploy at (2, 5) and so on.

Is that enough? As the above simulation showed, sometimes it requires 120 deployments to prepare entire 3 x 3. So in the worst case $23 \times 120 = 2760$ deployments to prepare entire 3 x 69 region which is more than our limit 1000. However, such worst case will not happen always. We ran another simulation to examine our new strategy:



The above figure shows how many of our 100000 simulations (y axis) required each possible number of deployments (x axis) to prepare all the cells inside 3 x 69 target region. As we can see, the maximum we needed is no more than 850 and our limit was 1000. So we can be confident that this strategy is good enough to pass test set 2.

There are many other strategies. One may be, to deploy the gopher to the cell that has the largest number of unprepared cells in 3 x 3 region centering at that cell. This strategy yields following simulation result (100000 runs), which looks better than the one before:



Analysis: Saving The Universe Again

Test set 1

Since there is at most one `C` instruction in this test set, we can solve the two cases independently.

If there is no `C` instruction in **P**, then none of our swaps will have any effect, so all we can do is check whether the damage of the beam exceeds **D**.

If there is one `C` instruction in **P**, then we can try every possible position for the `C` instruction in the program. Assuming that there is at least one position for the `C` instruction that causes the total damage not to exceed **D**, we can choose the scenario that requires the fewest swaps; the number of required swaps for a scenario is equal to the distance between the original and final positions of the `C` instruction.

Test set 2

To solve test set 2, we will first derive a formula to compute the total damage based on the positions of the `C` and `S` instructions in **P**. Let N_C and N_S be the number of `C` and `S` instructions in **P**, respectively. Let C_i be the number of `S` instructions to the right of the i -th `C` instruction, where i uses 1-based indexing.

Note that the i -th `C` instruction will increase the damage of the subsequent beams by 2^{i-1} . For example, in the input program `CSSCSCSS`, initially, all of the `S` instructions will inflict a damage of 1. Consider the damage dealt by the last `S` instruction. Since the robot has been charged twice, the damage output by the last instruction will be 4. Alternatively, we see that the damage, $4 = 1$ (initial damage) + 2^0 (damage caused by the first `C`) + 2^1 (damage caused by the second `C`). By breaking down the damage by each `S` instruction in the same manner, the total damage output, D , of the input program is given by:

$$D = N_S + C_1 \times 1 + C_2 \times 2 + \dots + C_{N_C} \times 2^{N_C - 1}.$$

Next, we investigate how each swap affects the amount of damage. A swap on adjacent characters which are the same will not affect the equation. When we swap the i -th `C` instruction with a `S` instruction to its right, the value of C_i will decrease by 1 since now there is one less `S` than before. On the other hand, swapping the i -th `C` instruction with an `S` instruction on its left will increase the value of C_i by 1. Note that in either case, we will only modify the value of C_i , and the other C values will remain the same. This suggests that we should only ever swap adjacent instructions of the form `CS`.

Therefore, executing M swaps is equivalent to reducing the values of C_i s such that the total amount of reduction across all C_i s is M . We want the total damage (according to the above equation) to be minimized. Clearly, we should reduce the values of C_i that contribute to the largest damage output, while making sure that each of the C_i s is nonnegative.

Intuitively, all of this math boils down to a very simple algorithm! As long as there is an instance of `CS` in the current program, we always swap the latest (rightmost) instance. After each swap, we can recompute the damage and check whether it is still more than **D**. If it is not, then we can

terminate the program. If we ever run out of instances of **CS** to swap, but the damage that the program will cause is still more than **D**, then the universe is doomed.

Analysis: Trouble Sort

Test set 1

Like bubble sort, Trouble Sort has $O(N^2)$ time complexity; the proof is explained below. With $N \leq 100$ for test set 1, we can run Trouble Sort to completion and simply iterate over the result list to find the first sorting error, if any (that is, a value that is greater than the value that follows it in the list).

Test set 2

Running $O(N^2)$ Trouble Sort to completion is too slow for $N \leq 10^5$.

Instead, let's break down what Trouble Sort is doing at each step. Let's consider an input list of 6 elements. Trouble Sort makes the following comparisons on each pass through the array:

- element 0 $\leftrightarrow$ element 2
- element 1 $\leftrightarrow$ element 3
- element 2 $\leftrightarrow$ element 4
- element 3 $\leftrightarrow$ element 5

Regardless of the length of the list, this table illustrates the fundamental flaw in Trouble Sort: even-index elements are compared with other even-index elements, and odd-index elements are compared with other odd-index elements, but even-index and odd-index elements are never compared with each other! This means that Trouble Sort is just bubble sort run separately on the even-index elements and the odd-index elements, interleaving them into the output list. Trouble Sort is correct only if interleaving the two sub-lists (the even-index list and the odd-index list) happens to produce another sorted list. Since there are $O(N)$ even-index and $O(N)$ odd-index elements, and since bubble sort is $O(N^2)$, Trouble Sort is also $O(N^2)$.

To solve test set 2, we can run our favorite $O(N \log N)$ sorting algorithm independently on the two sub-lists described above, interleave the sorted sub-lists, and then find the first sorting error as in our solution for test set 1.